

Rapport de Soutenance

fishotgun

Par Rabbit Hole

Adrien GAYERIE

Coordinateur de l'équipe, Graphiste, Développeur

Aurélien LAMARQUE

Coordinateur Adjoint, Développeur

Noah FAUVEL

Développeur

Joël LIPSKI

Développeur

Soutenance 2 – Vendredi 20 mars 2026

Sommaire

1	Introduction	2
2	Avancements	3
2.1	Menu	3
2.1.1	Bilan Aurélien.....	3
2.1.2	Bilan Joël	4
2.2	Physique	5
2.2.1	Bilan Noah	6
2.3	Network.....	6
2.3.1	Bilan Aurélien.....	7
2.4	IA des poissons	8
2.4.1	Bilan Adrien	9
2.4.2	Bilan Noah	9
2.5	Pêche	9
2.5.1	Bilan Joël	10
2.6	Assets	14
2.6.1	Bilan Adrien	14
2.6.2	Bilan Aurélien.....	16
2.7	Site Web	17
2.7.1	Bilan Etane	18
2.8	Tchat	18
2.8.1	Bilan Aurélien.....	18
3	Futures Tâches	20
3.1	Menu	20
3.1.1	Bilan Aurélien.....	20
3.2	Pêche	20
3.3.1	Bilan Joël	20
3.3	Inventaire	20
3.3.1	Bilan Joël	21
3.4	Boutiques.....	21
3.4.1	Bilan Noah	21
3.5	Organisation	21
4	Déroulement et atteinte des objectifs	21
5	Conclusion.....	22
	Plan de soutenance :	22

Annexe.....	22
-------------	----

1 Introduction

2 Avancements

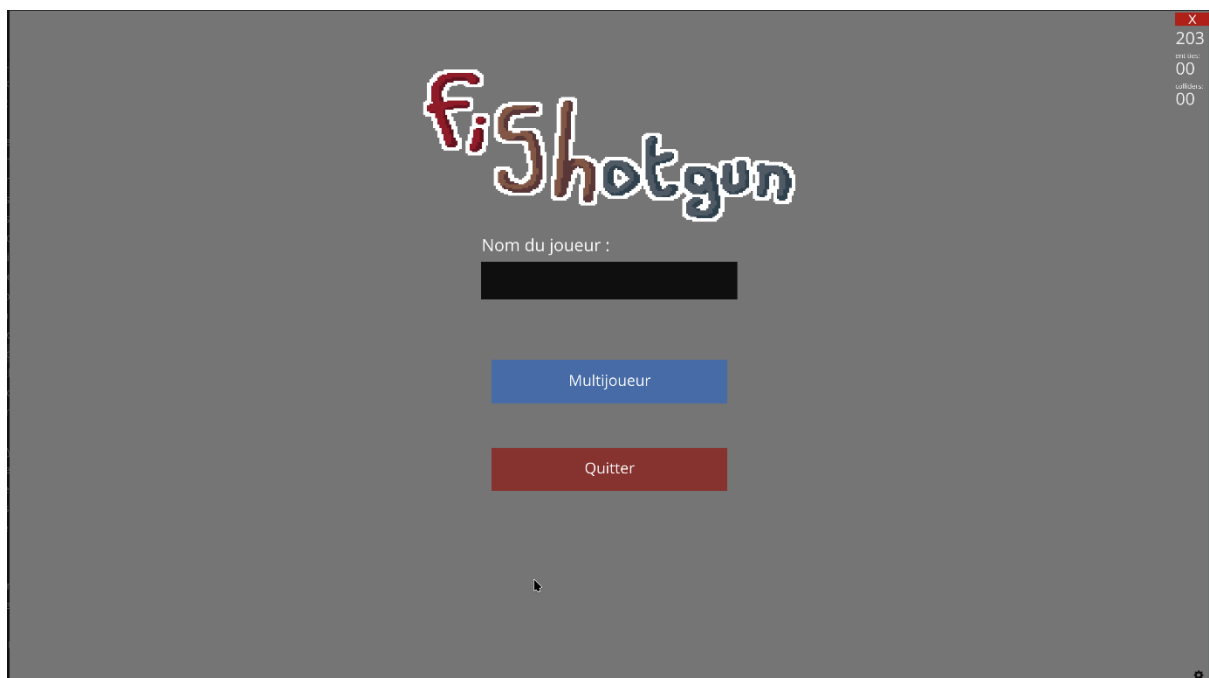
2.1 Menu

2.1.1 Bilan Aurélien

J'ai avancé sur les menus avec notamment la création du system de mise en place de menu simple. J'ai mis en place une variable global correspondant au menu qui est affiché et deux méthode **hide** et **show** qui permettent soit de cacher les menus ou d'afficher d'autres menus.

Chaque menu est fait à partir de la classe **Menu** à laquelle on lie les différents éléments du menu comme les boutons. Aussi j'ai créé une class spécifique pour faire des boutons qui change de menu afin de faciliter cette tâche.

Également le system de menu est constitué d'un **registre** permettant d'accéder à un menu à partir de son nom. Le menu principal est constitué d'un bouton multijoueur et d'un bouton quitter et du logo du jeu avec également un Input Field pour pouvoir rentrer son nom d'utilisateur.



J'ai également dû implémenter un système de liste de boutons, car Ursina n'en propose pas par défaut. Pour cela, j'ai créé une liste de boutons placés les uns à la suite des autres, puis j'ai utilisé un shader afin de n'afficher les boutons que dans une zone précise

de l'écran. Cela permet d'avoir un nombre potentiellement illimité de boutons qui peuvent être affichés dans cet espace, de manière similaire à une liste défilante.

J'ai toutefois rencontré quelques difficultés lors de cette implémentation, notamment à cause de problèmes de compatibilité avec OpenGL dû au fait que je développe sous une machine virtuelle linux sur mon mac qui ne supporte pas bien OpenGL.

2.1.2 Bilan Joël

J'ai contribué au développement du menu, ainsi je vais partager ce que j'ai réalisé de mon côté, qu'Aurélien a un peu modifié par la suite pour embellir notre interface.

Le système de menus repose sur une classe «Menu» dans «menu.py» qui hérite d'«Entity» d'Ursina, avec «parent=camera.ui» pour que les éléments soient toujours affichés en «par-dessus» la scène 3D (devant notre écran). Un dictionnaire global «_menus» référence tous les menus enregistrés par leur identifiant string, et deux fonctions «show()» et «hide()» gèrent l'activation et la désactivation des menus.

La fonction «show()» désactive le menu courant avant d'en activer un nouveau, et déverrouille la souris («mouse.locked = False», «mouse.visible = True»). La fonction «hide()» fait l'inverse et reprend le jeu via «application.resume()».

La navigation est gérée par des boutons classiques d'Ursina avec des callbacks «on_click» assignés à des méthodes. La classe «LinkingButton» hérite de «Button» et prend un menu cible en paramètre, appelant «show()» au clic.

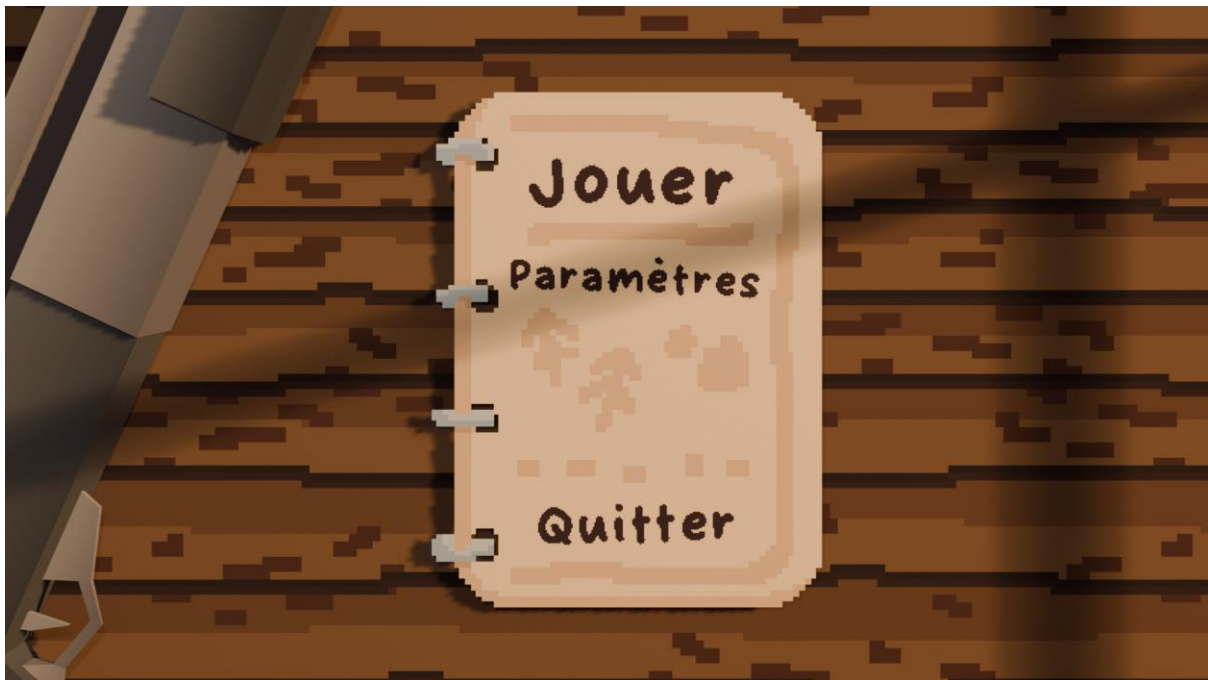
La navigation se présente comme cela: «MainMenu» → «PlayMenu» → soit «ServerListMenu» pour rejoindre un serveur existant, soit «HostMenu» pour en créer un. Un bouton retour est présent sur chaque écran pour revenir en arrière sans se perdre dans la navigation.

Le «MainMenu» affiche le titre «FISHOTGUN» en bleu ciel ainsi qu'un champ de saisie pour entrer son pseudo. Ce champ hérite d'«InputField» d'Ursina et surcharge «update()» pour que le champ ne soit actif que quand son menu parent est affiché. Le pseudo saisi est stocké dans une variable globale «name» au moment où le joueur clique sur «Multijoueur», via une méthode «switch()» qui wrap le callback original du bouton.

Le «ServerListMenu» affiche la liste des serveurs ajoutés manuellement par le joueur via «AddServerMenu», où il entre une adresse au format «ip:port». Les serveurs ajoutés apparaissent sous forme de «ServerButton» dans une liste scrollable, le scroll est géré en déplaçant un «Entity» parent «button_list» verticalement, avec un clamp pour éviter que la liste parte trop loin.

Un système de shader GLSL clippe visuellement les boutons qui dépassent de la zone de liste. Un premier clic sur un serveur le sélectionne, un deuxième clic déclenche la connexion via «world.join_world()».

Au niveau design on n'a pas encore intégré nos textures personnalisées dans le jeu, donc on a opté pour un côté minimaliste pour le moment. Le design final consistera d'un menu sous forme de carnet posé sur une table de pique-nique, avec le fusil à pompe du joueur à côté.



2.2 Physique

2.2.1 Bilan Noah

J'ai énormément travaillé sur la physique du jeu depuis la dernière soutenance. En effet, lors de la dernière soutenance, nous vous avons montré un jeu avec une physique cohérente et satisfaisante. Cependant, nous n'avons pas abordé les collisions et, lorsque nous avons commencé à les implémenter, nous nous sommes rendu compte que la manière dont nous gérions la physique était incompatible avec celles-ci. Nous avons donc dû faire machine arrière.

Nous avons d'abord regardé s'il était possible de revenir à la première méthode, c'est-à-dire utiliser le moteur Ursina pour gérer la physique ainsi que les collisions. Après avoir beaucoup travaillé dessus, nous avons constaté qu'Ursina gérait très mal les collisions.

Avec Aurélien, nous avons donc cherché une autre solution. Nous avons décidé d'utiliser Bullet, directement intégré dans Ursina, qui a la particularité de très bien gérer les collisions et la physique du jeu. Cependant, pour cela, toute la physique doit être attribuée uniquement à Bullet, sans intervention d'Ursina. Il faut donc pratiquement séparer Ursina et Bullet.

C'est, je pense, ce qui m'a pris le plus de temps, car une fois que j'ai compris comment fonctionnait Bullet, il n'était pas très difficile de créer une physique avec des collisions, notamment au sol. Cependant, l'intégrer dans notre projet et remplacer tout ce qui était géré par Ursina a été plus complexe. J'ai donc dû corriger les erreurs une par une et tout revoir.

Par exemple, une erreur qui m'a pris du temps à identifier venait du fait que certaines classes héritaient de la classe Entity, qui pouvait attribuer une sorte de physique aux objets ou au personnage, ce qui créait un conflit empêchant Bullet de fonctionner correctement.

J'ai également dû positionner notre personnage sur un bloc de terre, car la carte, qui est en cours de développement, ne possède pas encore de collisions, ce qui m'empêchait de tester mon code.

J'ai finalement réussi à terminer cette partie, mais pour l'instant, la physique et les collisions restent sur une branche différente de la branche main de notre projet. En effet, il subsiste de petits conflits entre la position estimée par le serveur et la position réelle calculée par le client lorsque la vitesse devient trop élevée, ce qui provoque des déplacements saccadés.

2.3 Network

2.3.1 Bilan Aurélien

Cette partie du projet est celle qui m'a pris le plus de temps en raison de sa complexité technique. J'ai commencé par améliorer le système de parsing des paquets réseau. L'objectif était d'optimiser la taille des données échangées entre le client et le serveur. Pour cela, j'ai ajouté la possibilité d'encoder la taille des données sur 1 ou 2 octets, en utilisant le premier bit comme indicateur (flag). Ce mécanisme permet aux petits objets d'utiliser seulement un octet pour stocker leur taille, tandis que les objets plus volumineux peuvent en utiliser deux, ce qui permet de représenter des tailles allant jusqu'à 32 768 bits. Cette optimisation réduit la taille moyenne des paquets et améliore l'efficacité globale des communications réseau.

Après cette amélioration, je me suis concentré sur une autre partie particulièrement complexe : la synchronisation des différents joueurs dans l'environnement multijoueur. Une approche simple aurait consisté à recevoir directement la position envoyée par chaque joueur et à la retransmettre aux autres. Cependant, cette méthode pose des problèmes importants, notamment en matière de sécurité et de cohérence, car un client pourrait envoyer n'importe quelle position et se téléporter librement dans le monde du jeu.

Pour éviter cela, j'ai mis en place un système de correction de mouvement côté serveur. Le principe est que le joueur n'envoie pas directement sa position, mais uniquement ses entrées de contrôle (inputs). Pour cela, j'ai créé une classe dédiée appelée `InputField`, qui utilise un masque de bits afin de représenter efficacement les différentes actions possibles du joueur. Cette approche permet de réduire la quantité de données envoyées dans les paquets tout en conservant les informations nécessaires à la simulation du mouvement.

Lorsque le serveur reçoit ces entrées, il met à jour l'état du joueur correspondant puis simule lui-même le mouvement attendu. Une fois la simulation effectuée, le serveur calcule la nouvelle position officielle du joueur et l'envoie à tous les clients connectés, y compris au joueur d'origine. De cette manière, le serveur reste l'autorité principale sur l'état du jeu.

Du côté des clients, le traitement diffère légèrement selon qu'il s'agit des autres joueurs ou du joueur local. Lorsqu'un client reçoit la position des autres joueurs, il interpole simplement leur position entre les différentes mises à jour envoyées par le serveur. Cette interpolation permet d'obtenir un mouvement visuellement fluide malgré le fait que les mises à jour réseau ne soient pas continues.

En revanche, la gestion de la correction pour le joueur local a été plus complexe à mettre en place. Le client du joueur fonctionne à la fréquence de rafraîchissement du jeu, soit environ 60 mises à jour par seconde, tandis que le serveur fonctionne à un rythme plus faible, d'environ 20 mises à jour par seconde. À cela s'ajoute la latence réseau, qui crée un décalage supplémentaire entre la position calculée côté client et celle validée par le serveur.

Pour résoudre ce problème, j'ai mis en place un système d'interpolation entre la position attendue côté client et celle renvoyée par le serveur. Cela permet de maintenir la synchronisation avec l'état officiel du serveur tout en évitant des corrections trop brutales qui pourraient provoquer des saccades perceptibles pour le joueur. Ainsi, les ajustements restent progressifs et le mouvement conserve une certaine fluidité.

Ce système pourrait encore être amélioré, notamment pour mieux gérer les situations de forte latence ou de perte de paquets réseau. Cependant, dans son état actuel, il offre déjà un fonctionnement satisfaisant pour les conditions d'utilisation prévues dans le projet.

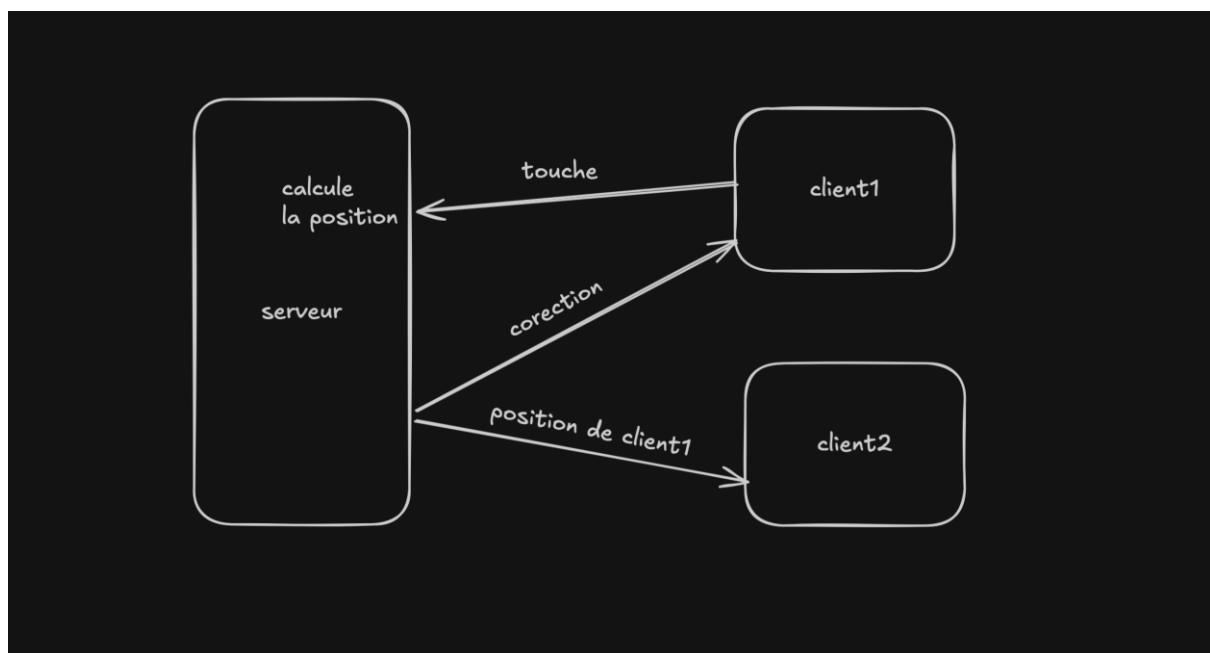


Schéma explicatif

2.4 IA des poissons

2.4.1 Bilan Adrien

Concernant l'IA des poissons, j'ai commencé par créer un environnement 3D afin de pouvoir après l'intégrer dans un environnement proche de celui en jeu. Après avoir discuté avec l'équipe de la manière dont se déplaceraient les poissons j'ai commencé à développer un algorithme de déplacement qui fonctionne ainsi.

Le poisson se déplace à une vitesse constante dans la direction où il regarde. Un point de passage est disposé à une position aléatoire sur le plan où est le poisson. Le choix du positionnement de ce point respecte des conditions qui le force à rester dans le cadre visible par le joueur. L'objectif du poisson est alors de passer par ce point. L'enchaînement de ces points constituera son trajet. Pour cela, le poisson tourne vers la gauche ou la droite (en fonction de ce qui est le plus rapide) afin de s'orienter vers le point. Le poisson avance alors vers le point en suivant des trajectoires arrondies qui rendent ses mouvements authentiques et légèrement difficiles à prévoir pour le joueur.

Nous avons cependant rencontré une difficulté. Le poisson, se déplaçant et s'orientant à vitesse constante, se retrouvait parfois à graviter autour du point de passage. Cela rendant la trajectoire cyclique, simple à anticiper et l'empêchant d'avancer vers un autre point. Pour répondre à ce problème nous avons modifier les mécaniques d'orientation du poisson. Le rapport vitesse de déplacement / vitesse de rotation a été ajuster pour que le poisson tourne plus vite qu'il n'avance.

Également, ce dernier détecte lorsqu'il est tourné à l'opposé du point du passage et adapte ça vitesse d'orientation en conséquence : moins le poisson est orienté vers le point, plus il tourne vite. Cela permet d'éviter les trajectoires parfaitement arrondies lorsque le poisson se retourne, toujours dans l'objectif d'obtenir un déplacement naturel.

2.4.2 Bilan Noah

En effet, Adrien parle très bien de cette partie. J'ai aidé Adrien sur la gestion des déplacements du poisson. C'est lui qui a eu l'idée initiale de générer des points aléatoires pour orienter le poisson. Une fois cette méthode implémentée, nous avons travaillé ensemble sur ses déplacements et avons obtenu un résultat très satisfaisant.

2.5 Pêche

2.5.1 Bilan Joël

Le système de pêche a bien évolué depuis la dernière soutenance, en effet il s'agit dorénavant d'une base solide à laquelle on pourra ajouter les fonctionnalités finales d'ici la dernière soutenance, il s'agit d'un mini-jeu complet intégré dans l'architecture multijoueur du projet.

Pour commencer, j'ai repris le fichier «fish.py» réalisé par Noah et Adrien sur l'IA des poissons afin de l'implémenter dans notre jeu.

Ainsi j'ai modifié légèrement quelques fonctionnalités telles que la méthode «look_at(target_pos)» qui gère à la fois le déplacement et la rotation du poisson vers un point cible, pour que ce soit applicable dans l'environnement du jeu.

J'ai surtout changé la manière dont les poissons se déplacent, dorénavant ils changent de cible vers laquelle ils nagent soit lorsqu'ils sont alignés parfaitement vers cette cible soit quand ils sont trop près de celle-ci, cela permet de rendre le mouvement des poissons plus fluide pour le joueur tout en évitant que le poisson tourne en rond en boucle pour toujours.

Une fois la mécanique de poisson validée, j'ai intégré tout ça dans le jeu principal via deux classes : «FishingSpot» dans «spot.py» et «FishingScene» dans «fish.py».

Le «FishingSpot» est une entité sphérique placée dans le monde 3D qui détecte la proximité du joueur dans «main.py» via la fonction «distance()» d'Ursina, quand le joueur est à portée et appuie sur «E» la fonction «enter_fishing()» est appelée.

La «FishingScene» est une classe Python pure, sans héritage d'«Entity», ce qui veut dire qu'elle n'est pas automatiquement mise à jour par Ursina, son «update()» est appelé manuellement depuis «main.py» à chaque frame, uniquement quand «fishing_scene.enabled» est «True».

Ce choix architectural garantit un contrôle total sur la boucle de la nouvelle scène.

Pendant la scène de pêche, le joueur du monde extérieur est désactivé via «player.disable()» et la souris est déverrouillée, ce qui empêche toute interaction avec le monde 3D principal.

Les entités créées pendant la pêche (eau, poissons, points, barres UI) sont toutes stockées dans une liste «_entities» et détruites proprement à la fin via «destroy()», bien sûr ils ne sont à aucun moment visibles pour les autres joueurs et sont mis réellement sous la carte par praticité.

Un des défis techniques a été de repositionner la caméra pour la vue de pêche sans perturber la caméra du «ThirdPersonController».

Dans Ursina la caméra est enfant du joueur dans la hiérarchie de scène, ce qui veut dire que modifier «camera.position» directement déplace la caméra par rapport au joueur et non dans l'environnement.

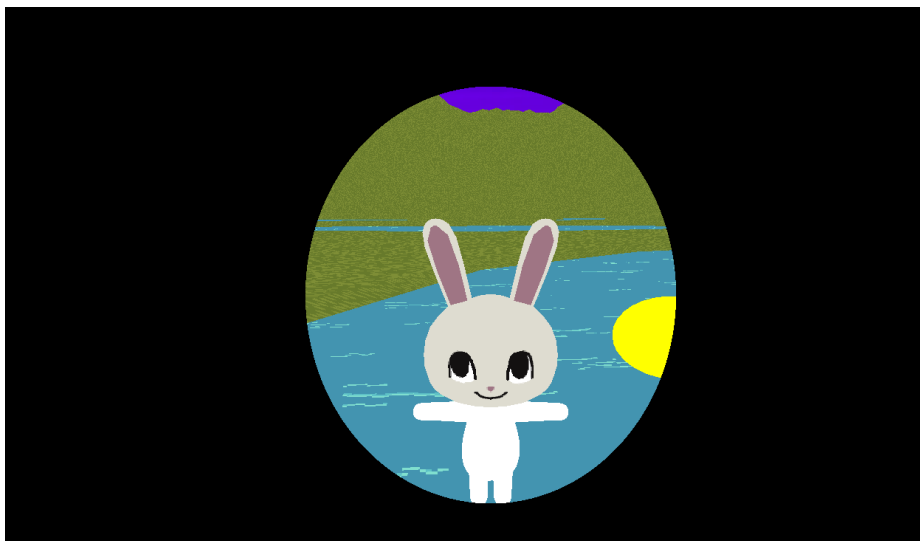
La solution a été de sauvegarder le parent original de la caméra puis de la reparenter à «scene», ce qui permet ensuite d'utiliser «camera.world_position» et «camera.world_rotation» pour la positionner librement.

La vue du dessus est obtenue avec «camera.rotation = (90, 0, 0)» et «camera.position = (0, -30, 0)», ce qui donne un décalage vertical négatif qui place toute la scène de pêche loin en dessous du monde de jeu principal, garantissant qu'aucune entité du jeu principal n'interfère visuellement.

Cette isolation est aussi importante dans le contexte multijoueur : les entités de la scène de pêche ne sont jamais envoyées au serveur, elles n'existent que localement sur la machine du joueur qui pêche.

La transition entre le monde de jeu et la scène de pêche est gérée par la classe «IrisTransition» dans «transitions.py».

Cette classe est appelée ainsi car il s'agit d'un effet «iris wipe» inspiré des dessins animés, où un cercle noir se rétrécit progressivement jusqu'à couvrir tout l'écran, puis se rouvre.



Techniquement cet effet est implémenté via un fragment shader GLSL appliqué à un quad plein écran attaché à «camera.ui».

Le shader calcule pour chaque pixel sa distance au centre de l'écran et utilise «discard» pour rendre transparent tout pixel dont la distance est inférieure au rayon courant «radius», créant ainsi un cercle «troué» dans le quad noir.

Le rayon est passé au shader comme uniform et animé en Python frame par frame, avec une correction du ratio d'aspect appliquée via «aspect/2» pour garantir que le cercle reste rond et non ovale.

La transition se déroule en trois phases gérées par une machine d'états avec les états «CLOSING», «BLACK» et «OPENING».

Le callback «on_black» est déclenché exactement au moment où l'iris est complètement fermé, c'est à ce moment que «fishing_scene.start()» est appelé, garantissant que le changement de caméra et de scène est invisible pour le joueur (car il se déroule pendant l'instant d'écran noir).

Au lancement de la scène, quatre poissons sont instanciés à des positions fixes avec des types choisis aléatoirement via «shuffle()». Chaque poisson reçoit un «on_click» assigné via une lambda avec argument par défaut. Cette syntaxe est nécessaire parce que les lambdas en Python capturent les variables par référence et non par valeur, donc sans l'argument par défaut toutes les lambdas de la boucle référenceraient le même poisson (la dernière valeur de la variable après la fin de la boucle).



Quand le joueur clique sur un poisson, «_on_fish_click» vérifie si un poisson est déjà sélectionné.

S'il ne l'est pas, «_select()» est appelé: pour chaque poisson non choisi, son point cible est détruit immédiatement et le poisson est ajouté à la liste «_fleeing» avec une direction de fuite calculée comme opposée au poisson sélectionné.

Les poissons en fuite sont mis à jour dans «update()» à une vitesse de 10 unités/seconde et détruits après 1.2 secondes via «invoke(..., delay=1.2)». Ce délai est choisi pour que les

poissons aient le temps de sortir du champ de vision avant de disparaître, donnant l'impression qu'ils s'échappent vraiment.

Trois types de poissons sont définis dans la classe «FishType» sous forme de dictionnaires Python : «WEAK» (5 PV, vitesse 1.5, taille 1.3), «NORMAL» (10 PV, vitesse 2, taille 1) et «STRONG» (20 PV, vitesse 2.5, taille 0.7). Ceci sont évidemment des types temporaires de test remplaçant les types «Abondant, Discret, Insaisissable».

Ces dictionnaires sont passés au constructeur de «Fish» via un «kwargs.pop('fish_type', FishType.NORMAL)», le «pop» est nécessaire car «fish_type» n'est pas un argument natif d'«Entity» et provoquerait une erreur si transmis à «super().init()».

Les quatre poissons spawned sont toujours «[WEAK, NORMAL, NORMAL, STRONG]» pour le moment, mélangés aléatoirement, garantissant une composition équilibrée mais avec une position imprévisible.

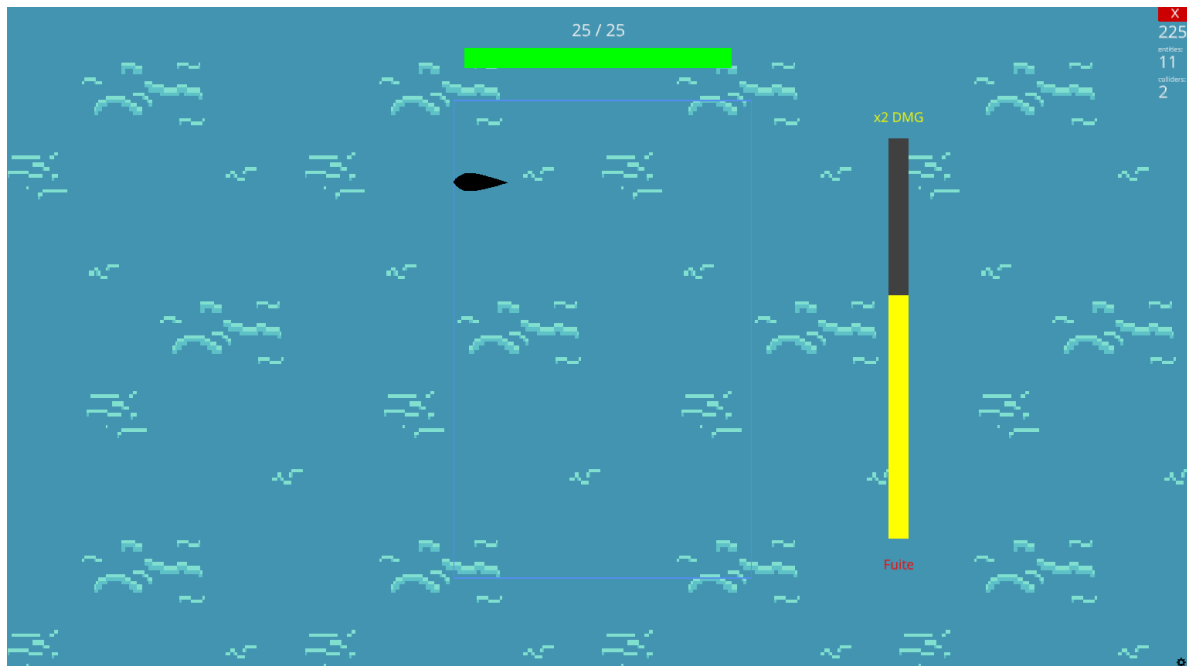
Quand le joueur clique sur le poisson sélectionné, «_deal_damage()» est appelé avec des dégâts de base de 1 («player_damage = 1»), doublés si la barre de pression dépasse 80%. Quand les PV atteignent 0, «request_stop()» est appelé automatiquement et la pêche s'arrête, le joueur reprend possession de son corps.

Deux barres d'interface ont été ajoutées après la sélection du poisson. La barre de PV est un quad horizontal en haut de l'écran avec «origin=(-0.5, 0)» pour ancrer le pivot au bord gauche, ce qui permet de réduire «scale_x» vers la droite naturellement sans recalcul de position.

De plus, sa couleur change dynamiquement: verte au dessus de 50% des PV, orange entre 25% et 50%, rouge en dessous, avec un texte qui affiche les PV numériques au format «X/max» juste au dessus.

La barre de pression est verticale, ancrée à droite de l'écran, avec des labels fixes «x2 DMG» en haut et «Fuite» en bas. Sa hauteur est mise à jour à chaque frame en fonction de «_pressure», une valeur entre 0 et 1.

Elle monte à +0.4/seconde quand la souris est à moins de 1.5 unités du poisson dans l'espace monde, détecté via «mouse.world_point» qui raycaste sur le collider de l'eau, et descend à -0.6/seconde sinon.



Cette asymétrie intentionnelle crée une tension et du stress pour le joueur, surtout contre un poisson fort: il faut maintenir activement la souris sur le poisson pour garder la pression haute. Un flag «_stopping» a été ajouté pour éviter que «request_stop()» soit appelé plusieurs fois dans la même frame, problème qui faisait que tout disparaissait avant que l'iris puisse se refermer.

Cela survenait car «update()» continuait à s'exécuter après le premier appel, le flag bloque immédiatement toute logique supplémentaire via un «return» en début de «update()»

2.6 Assets

2.6.1 Bilan Adrien

Comme vous pourrez le voir sur le tableau de déroulement page 21, nous avons bien avancé sur la création d'assets pour le jeu. La création de différentes textures constitue en grande partie cet avancement. En utilisant Photoshop, j'ai pu faire des textures pour : le bois, l'eau, l'herbe, le menu... Ces textures sont visibles en annexe de ce rapport. Elles ont été faites et pensé en équipe, que ce soit en appel avec aurélien ou abordé en réunion comme pour le menu.

Cela nous a également permis de pouvoir pointer plus distinctement la direction artistique choisie pour le jeu.

Également, pour ce qui est de la partie 3d, j'ai pu travailler sur la conception et l'application d'une texture sur le model du joueur. Je me suis ensuite directement

attelé à créer un modèle pour le fusil à pompe, élément central de notre jeu. Toujours en utilisant Blender, j'ai pu concevoir le modèle en m'appuyant sur des images de référence est lui apposer une texture que j'ai fait. Ce modèle a déjà été utilisé, notamment dans la scène de menu de démarrage comme il est possible de le constater dans la partie 2.1.2 de Joël. Il reste encore associer le modèle du joueur avec celui du fusil.

Aurélien s'est occupé d'appliquer les textures sur le model du terrain.

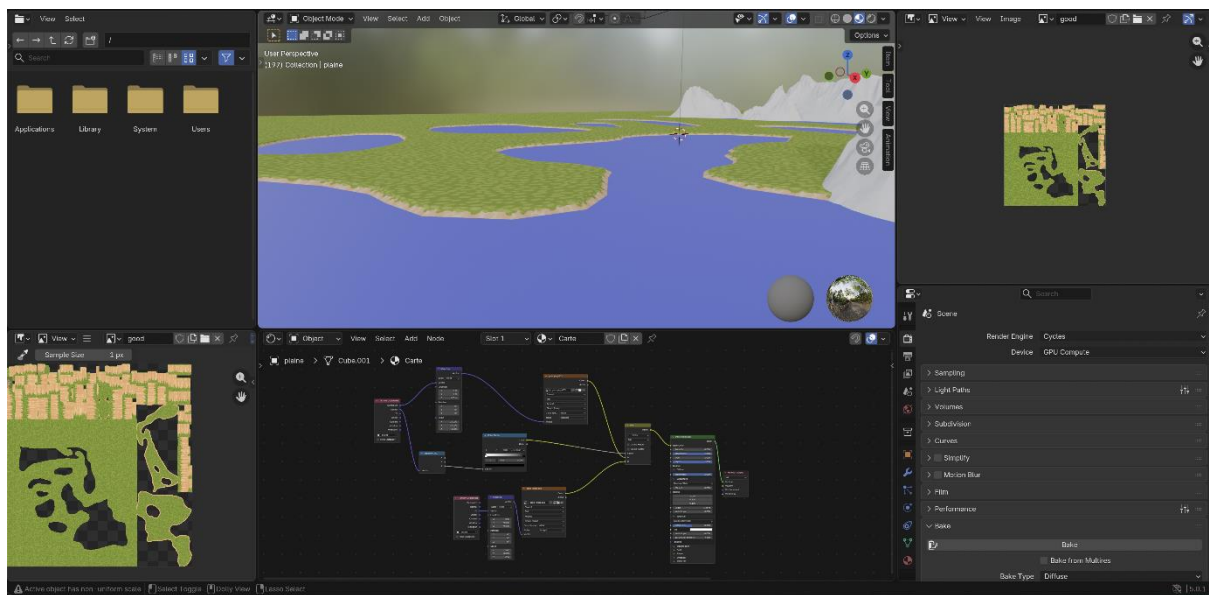
Enfin, concernant l'environnement sonore, rien n'a été ajouté. Les musiques ont été finies avant la première soutenance. Quant aux bruitages et autres effets sonores, nous travaillerons dessus dans le but de rendre l'expérience de jeu la plus immersive possible.

2.6.2 Bilan Aurélien

Je me suis ensuite occupé de la partie *world* du jeu. La première étape a consisté à intégrer directement dans le jeu, avec Ursina, le modèle 3D du terrain réalisé par Adrien ainsi que ses textures de base. L'objectif était d'avoir rapidement une première version fonctionnelle du monde en jeu, afin de vérifier que le modèle s'affichait correctement et que les textures principales étaient bien appliquées sur le terrain.

Une fois cette intégration réalisée, je me suis concentré sur l'amélioration du rendu visuel et sur la préparation des textures dans Blender. J'ai notamment travaillé sur l'UV mapping du terrain afin d'avoir un meilleur contrôle sur la manière dont les textures sont appliquées sur la géométrie. Cette étape est importante pour éviter les déformations de texture et obtenir un rendu plus propre, même si cette version retravaillée n'est pas encore entièrement importée dans le jeu.

Comme on peut le voir sur l'image, le matériau est composé de plusieurs nodes qui permettent de mélanger les textures et de contrôler leur affichage sur le terrain.



Le principe consiste à utiliser deux textures principales : une texture d'herbe pour le sol et une texture différente, notamment du sable, pour les zones de transition comme les bords proches de l'eau ou certaines pentes du terrain.

Pour déterminer quelle texture doit être affichée, j'utilise la normale de la surface du terrain. Cela permet d'adapter automatiquement la texture en fonction de l'orientation des faces.

J'ai également ajusté l'échelle des textures afin que le motif reste cohérent visuellement et ne paraisse pas trop répétitif sur les grandes surfaces.

Cependant, une contrainte technique s'est posée : le moteur Ursina ne supporte pas facilement l'utilisation de plusieurs textures combinées dans un même matériau. Il a donc fallu trouver une solution pour conserver le rendu tout en restant compatible avec le moteur.

Après m'être renseigné, j'ai utilisé une fonctionnalité de Blender appelée *baking*. Pour cela, j'ai créé une nouvelle UV map dédiée au terrain, puis j'ai généré une texture finale à partir du rendu complet du matériau. Concrètement, Blender calcule le résultat du mélange des différentes textures et l'enregistre dans une seule image. Cette texture peut ensuite être utilisée directement dans le jeu.

Cette méthode correspondait bien à mon besoin, car elle permet de garder le rendu visuel préparé dans Blender tout en utilisant une seule texture dans Ursina.

2.7 Site Web

2.7.1 Bilan Etane

Pour ma part, je me suis occupé du site vitrine de Fishotgun.

J'ai utilisé une stack web simple en trois parties.

D'abord, HTML, qui sert à faire la structure du site. C'est ce qui permet d'organiser les différentes sections, comme la bannière, la présentation du jeu, l'équipe ou la roadmap.

Ensuite, TailwindCSS, qui sert à gérer l'apparence du site. C'est grâce à ça que j'ai pu travailler la mise en page, les couleurs, les espacements, les cartes, les effets visuels et l'ambiance générale du site.

Enfin, JavaScript, qui sert à rendre le site interactif. Par exemple, c'est ce qui permet de gérer le changement de langue, le lecteur de musique, le carrousel d'images, ou encore les effets au survol dans la section équipe.

Donc, pour résumer très simplement :

HTML = la structure

TailwindCSS = le style

JavaScript = l'interactivité

À partir de cette base, j'ai retravaillé le site pour qu'il soit plus lisible, plus cohérent visuellement, et plus représentatif de l'univers de Fishotgun. J'ai aussi amélioré les textes, surtout en français, et réorganisé le code JavaScript en plusieurs fichiers pour qu'il soit plus propre et plus facile à maintenir.

2.8 Tchat

2.8.1 Bilan Aurélien

Je me suis chargé de la mise en place du tchat. J'ai d'abord implémenté la logique côté serveur : le joueur envoie un paquet contenant son message au serveur, et celui-ci se contente ensuite de diffuser ce message à l'ensemble des autres joueurs.

La partie la plus complexe a été la conception du menu de chat lui-même, notamment en raison du défi que représentait la création d'une interface personnalisée avec Ursina.

Pour l'affichage des différents messages, j'ai mis en place un fond gris sur lequel vient se superposer une liste de messages. Ceux-ci sont affichés en utilisant la classe Text d'Ursina, avec les noms des joueurs colorés en orange pour une meilleure lisibilité.

Concernant la saisie de texte, j'ai réutilisé le composant InputField d'Ursina. Je l'ai adapté afin de n'afficher qu'une partie du texte complet en surchargeant la méthode render. Cela permet d'écrire des messages longs sans qu'ils ne dépassent visuellement le cadre défini.

J'ai toutefois rencontré certaines complications, notamment avec l'affichage du texte dans Ursina, qui applique un contour de manière irrégulière. Ne pouvant pas réellement modifier le moteur et n'ayant pas trouvé de solution satisfaisante à ce problème, j'ai choisi de poursuivre le développement sur d'autres aspects du projet.



Rendu du chat

3 Futures Tâches

3.1 Menu

3.1.1 Bilan Aurélien

Le menu marche correctement, nous allons surtout implémenter nos assets personnalisés dans celui-ci dans un futur proche.

Ainsi on va devoir trouver une solution pour que le menu marche en «3D» car le carnet posé sur la table le sera. Soit nous ferons un faux carnet invisible qui sera en 2D sur l'UI, pour faciliter les clics, soit à l'aide d'un raycast de la souris dans l'environnement, on réfléchit encore à laquelle de ces deux solutions sera meilleure.

3.2 Pêche

3.3.1 Bilan Joël

Pour ce qui est de la pêche, la mise en place d'un système de collection du poisson est en cours, afin de permettre au joueur de récupérer le poisson après l'avoir battu ou le perdre lors d'une fuite.

De plus, plusieurs fonctionnalités seront ajoutées telles que les types/raretés de poissons, qui vont modifier leurs statistiques permettant un Game Play varié.

Ces poissons pourront ensuite être vendus pour un certain prix en boutique, ainsi il faudra leur donner un multiplicateur d'argent en fonction de leur taille aussi.

3.3 Inventaire

3.3.1 Bilan Joël

L'inventaire n'est actuellement pas mis en place, mais ce sera un de nos plus gros défis prochainement étant donné qu'il y'a beaucoup de code derrière celui-ci.

Ainsi, on compte réaliser un inventaire qui servira à stocker et montrer les poissons capturés par le joueur, et afficher le prix qu'ils vaudront lors de leur vente.

On réfléchit à la mise en place d'informations par rapport aux poissons dans l'inventaire, mais il sera probablement plus pratique d'afficher seulement leur prix dans l'inventaire quand on passe la souris dessus, et afficher les vraies statistiques des poissons dans le FishoDex qu'on va bientôt mettre en place.

3.4 Boutiques

3.4.1 Bilan Noah

Nous n'avons pas pu commencer l'implémentation de la boutique qui était prévue. Nous comptons créer une boutique proposant des améliorations de notre gameplay, ainsi que des achats possibles, notamment des munitions ou un nouveau Shotgun. Nous sommes donc en retard sur ce point, car pour ma part, la physique m'a pris beaucoup plus de temps que prévu.

3.5 Organisation

Nous avons pu constater un retard dans l'avancement du projet, en partie dû à un suivi des tâches imprécis, ce qui a ralenti la production. De plus, de nombreux problèmes imprévus se sont présentés au cours du développement, entraînant des ajustements constants et ralentissant davantage la progression et la mise en production du projet.

Beaucoup de problèmes ainsi qu'une méconnaissance du moteur ont également fait que beaucoup de temps a été investi dans la recherche et la correction plutôt que dans l'ajout pur de fonctionnalités. Lors de notre dernière réunion, nous avons justement revu nos objectifs à la baisse afin de pouvoir livrer un projet fini dans les temps, en nous concentrant sur la pêche et les fonctionnalités principales du jeu.

4 Déroutement et atteinte des objectifs

Dans le tableau suivant vous pourrez y retrouver les différents avancements qui ont été faits avec leur pourcentage relatif.

Tâches	Soutenance 1	Différence	Avancée actuelle
Assets	70%	+30	La plupart des assets personnalisés est faite, il manque cependant leur implémentation dans le jeu.
Pêche	70%	0	Entrée en combat, système de dégâts et de fuite du poisson, barre de vie et barre de pression tous intégrés.
Site Web	50%	0	Le site web a un peu avancé, malgré la perte d'un de nos membres.
Mouvements	80%	-20	Il manque uniquement la synchronisation avec le serveur On a rencontré de nombreux soucis face à la physique du jeu, mais on s'en sort.
Interactions multijoueur (tchat)	50%	+10	Le tchat a été mis en place et est fonctionnel. Il manque seulement les animations type émotes.
Networking	100%	0	Le multijoueur est fonctionnel, on peut voir les autres joueurs en jeu.
Interface	80%	0	Base de Menu solide, avec quelques éléments d'interface en cours de création qui seront bientôt intégrés.
IA des poissons	80%	+10	Mouvement des poissons fluidifié, détections supplémentaires ajoutées pour éviter qu'ils tournent en rond.
Camera	100%	0	La caméra fonctionne sans aucun soucis.
Carte (environnement)	70%	+10	Implémentation de la carte avec les différent lac et terrain ajoutés au jeu.
Boutique	0%	-60	Le fonctionnement des boutiques n'a pas encore été mis en place.
Marketing (boîtier, entreprise...)	50%	0	Le site web présente bien tout le jeu, malgré qu'il ne soit encore pas terminé. Aussi des recherches ont été faites concernant le nom de l'entreprise.

Avance	Retard	Total
+60	-80	-20 (en retard)

Comme il est possible de le constater, les progrès qui ont été faits ne correspondant pas parfaitement aux prévisions de début de projet, le bilan global des avancements est soumis à un certain retard.

Cela s'explique notamment par le fait que la boutique n'a pas encore été développée. Nous avons préféré mettre l'axe sur des mécaniques qui nous semblait plus essentielles

pour le jeu comme les assets ou les interactions multijoueur. Cela est en partie à l'origine de ce décalage.

Pour ce qui est du site web, son avancée a été limitée car un de nos membres a quitté EPITA et il était le responsable de cette tâche, mais il avait déjà avancé dessus de son côté et nous a partagé ce qu'il a fait avant de partir, nous permettant d'atteindre l'objectif attendu.

Néanmoins nous gardons en tête ces points sur lesquels nous travaillerons pour la soutenance finale. Nous souhaitons toujours, à terme, parvenir à développer un jeu complet, à la hauteur des objectifs que nous nous étions fixés.

La mécanique principale du jeu, la pêche, a bien été entamée, il suffit de lui ajouter et lui lier les fonctionnalités telles que la boutique, l'inventaire et le système d'argent.

Ainsi le développement du jeu avance bien, on a plusieurs bases sur lesquelles s'appuyer pour développer tout notre jeu et on arrive à résoudre les problèmes rencontrés lors du développement.

D'ici la dernière soutenance, tout devrait être fini si l'on ne rencontre pas de gros problèmes sur le chemin.

5 Conclusion

Cette seconde soutenance nous a permis de présenter l'évolution concrète du projet **Fishotgun** et de montrer les bases solides qui ont été mises en place depuis la première présentation. Une grande partie du travail s'est concentrée sur les fondations techniques du jeu : le système de menus, la physique avec gestion correcte des collisions, le fonctionnement du multijoueur, ainsi que l'intégration des premières mécaniques de gameplay comme la pêche et l'IA des poissons. Ces éléments constituent désormais une base stable sur laquelle nous pouvons continuer à construire les fonctionnalités finales.

Nous avons également avancé sur l'intégration du monde de jeu, des assets et du site web, ce qui permet progressivement de donner une identité visuelle et une cohérence globale au projet. En parallèle, certaines fonctionnalités importantes comme le tchat multijoueur ont été mises en place et fonctionnent déjà en jeu, ce qui confirme la viabilité de l'architecture choisie.

Cependant, cette phase de développement a aussi mis en évidence plusieurs difficultés techniques et organisationnelles. Certains choix ont dû être revus, notamment concernant la physique et les collisions, ce qui a entraîné un travail supplémentaire et un léger retard sur le planning initial. De plus, certaines parties du projet, comme la boutique ou l'inventaire, restent encore à développer.

Pour la prochaine étape, notre objectif sera donc de finaliser les systèmes encore manquants, améliorer la stabilité du multijoueur et intégrer les éléments visuels définitifs du jeu. Nous allons également renforcer l'organisation du suivi des tâches afin d'optimiser l'avancement global du projet.

Malgré le retard constaté, le projet progresse dans la bonne direction et les bases actuelles nous permettent d'envisager la soutenance finale avec un jeu plus complet, cohérent et jouable.

Plan de soutenance :

Introduction : Noah – 30s

- I) Network
Aurélien – 1min30
- II) Menu
Aurélien – 1min30
- III) Physique
Noah - 2min30
- IV) Pêche
Joël - 3min
- V) Assets
Adrien 2min30

Conclusion : Adrien – 30s

Annexe